SCHOOL OF ENGINEERING
JAWAHARLAL NEHRU UNIVERSITY

Project Report
(Data Structure)

Topic: <u>Inventory Management System</u>

Submitted by:

Harshit Tyagi (25/11/EC/014)

Saransh Singh (25/11/EE/008)

Inventory Management System using Hash Map

Github Link: https://github.com/saransh1274/Inventory-Management-System.git

Objective:

To develop an efficient store inventory management system that enables quick product search, addition, deletion, and stock updates using a hash map for fast data access.

Data Structure Concepts Used:

- **Hash Map** – for Fast insertion, search, deletion, updating
- **Structure (struct in C++)** - It helps organize product information in a clean and structured way.
- **Unordered Map** - It provides built-in hashing and fast access without manually creating a hash table.
- **Traversal** - To go through all stored products one by one.
- **Conditional Checking** - This helps in decision-making inside the system.

Expected Outcome:

The expected outcome of this project is to develop an efficient and reliable inventory management system for stores using a hash map. The system should allow quick addition, deletion, searching, and updating of product details with minimum time complexity. The project demonstrates the practical use of data structures in solving real-world business problems.

Features Implemented:

- Add Product
- Search Product
- Delete Product
- Update Stock
- Display All Products
- Total Inventory Value
- Update Product Details

Work Distribution between Team Members

1. Saransh Singh

- ✓ **Add Product** – This function is implemented for taking product details such as Product ID, name, quantity, and price from the user. These details are stored in the unordered_map using Product ID as the key and the product structure as the value.

- ✓ **Search Product -** This function was implemented using Product ID as input from the user. The program checks whether the key exists in the hash map and displays the corresponding product details if found.

- ✓ **Delete Product** – This function was implemented using the erase() operation of unordered_map. The user enters the Product ID, and if it exists, the product is removed from the inventory.

2. Harshit Tyagi

- ✓ **Update Product** – This function works by first asking the user to enter the Product ID of the item that needs modification. The program searches for that Product ID in the unordered_map; if found, it updates the required fields such as quantity or price by directly changing the stored values in the product structure.

- ✓ **View All Product –** This function displays all the products currently available in the inventory. Internally, the program uses a loop to traverse the entire unordered_map and prints each product's ID, name, quantity, and price in a tabular format for easy understanding.

- ✓ **Total Inventory Value –** This function calculates the total monetary value of all products present in the inventory. It works by traversing all products and multiplying each product's quantity by its price, then adding all these values together to get the final total inventory value.

- ✓ **Update Product Details –** This function is used when complete product information such as name, quantity, and price needs to be changed. The program first searches the product using Product ID and, if found, replaces the old details with the new values entered by the user, ensuring updated and accurate inventory records.

Code:

```
#include <iostream>
#include <unordered_map>
using namespace std;

struct Product {
    int id;
    string name;
    int quantity;
    float price;
};

int main() {
    unordered_map<int, Product> inventory;
    int choice;

    do {
        cout << "\n--- Inventory Management System ---\n";
        cout << "1. Add Product\n";
        cout << "2. Search Product\n";
        cout << "3. Delete Product\n";
        cout << "4. Update Stock\n";
        cout << "5. Display All Products\n";
        cout << "6. Total Inventory Value\n";
        cout << "7. Update Product Details\n";
        cout << "0. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        int id, qty;
        string name;
        float price;
```

```
switch (choice) {

    case 1:
        cout << "Enter ID: ";
        cin >> id;

        if (id < 0) {
            cout << "Invalid ID!\n";
            break;
        }

        if (inventory.find(id) != inventory.end()) {
            cout << "Product already exists!\n";
            break;
        }

        cout << "Enter Name: ";
        cin >> name;

        cout << "Enter Quantity: ";
        cin >> qty;
        if (qty < 0) {
            cout << "Invalid quantity!\n";
            break;
        }

        cout << "Enter Price: ";
        cin >> price;
        if (price < 0) {
            cout << "Invalid price!\n";
            break;
        }

        inventory[id] = {id, name, qty, price};
        cout << "Product added successfully!\n";
        break;
```

```
case 2:
        cout << "Enter ID to search: ";
        cin >> id;

        if (inventory.find(id) != inventory.end()) {
            Product p = inventory[id];
            cout << "ID: " << p.id << endl;
            cout << "Name: " << p.name << endl;
            cout << "Quantity: " << p.quantity;
            if (p.quantity < 5)
                cout << " ⚠ LOW STOCK";
            cout << endl;
            cout << "Price: " << p.price << endl;
        } else {
            cout << "Product not found!\n";
        }
        break;

    case 3:
        cout << "Enter ID to delete: ";
        cin >> id;

        if (inventory.erase(id)) {
            cout << "Product deleted!\n";
        } else {
            cout << "Product not found!\n";
        }
        break;
```

```
case 4:
        cout << "Enter ID: ";
        cin >> id;

        if (inventory.find(id) != inventory.end()) {
            cout << "Enter quantity to add/remove: ";
            cin >> qty;

            if (inventory[id].quantity + qty < 0) {
                cout << "Invalid operation! Quantity cannot be negative.\n";
                break;
            }

            inventory[id].quantity += qty;
            cout << "Stock updated!\n";
        } else {
            cout << "Product not found!\n";
        }
        break;

    case 5:
        if (inventory.empty()) {
            cout << "No products available.\n";
        } else {
            for (auto &item : inventory) {
                Product p = item.second;
                cout << "\nID: " << p.id << endl;
                cout << "Name: " << p.name << endl;
                cout << "Quantity: " << p.quantity;
                if (p.quantity < 5)
                    cout << " ⚠ LOW STOCK";
                cout << endl;
                cout << "Price: " << p.price << endl;
            }
        }
        break;
```

```
 case 6: {
        float total = 0;
        for (auto &item : inventory) {
            total += (item.second).price * item.second.quantity;
        }
        cout << "Total Inventory Value: " << total << endl;
        break;
    }

    case 7:
        cout << "Enter ID: ";
        cin >> id;

        if (inventory.find(id) != inventory.end()) {
            cout << "Enter new name: ";
            cin >> name;

            cout << "Enter new price: ";
            cin >> price;

            if (price < 0) {
                cout << "Invalid price!\n";
                break;
            }

            inventory[id].name = name;
            inventory[id].price = price;

            cout << "Product details updated!\n";
        } else {
            cout << "Product not found!\n";
        }
        break;

    case 0:
        cout << "Exiting...\n";
        break;

    default:
        cout << "Invalid choice!\n";
    }

  } while (choice != 0);

  return 0;
}
```

Function Selector:-

```
--- Inventory Management System ---
1. Add Product
2. Search Product
3. Delete Product
4. Update Stock
5. Display All Products
6. Total Inventory Value
7. Update Product Details
0. Exit
Enter choice:
```

Add Product :-

```
Enter choice: 1
Enter ID: 101
Enter Name: Mango
Enter Quantity: 51
Enter Price: 50
Product added successfully!
```

Search Product:-

```
Enter choice: 2
Enter ID to search: 102
ID: 102
Name: Banana
Quantity: 25
Price: 12
```

Delete Product :-

```
Enter choice: 3
Enter ID to delete: 101
Product deleted!
```

Display all Product:-

```
Enter choice: 5


ID: 102
Name: Banana
Quantity: 37
Price: 12
```

Update Product :-

```
Enter choice: 4
Enter ID: 102
Enter quantity to add/remove: 12
Stock updated!
```

Total inventory value :-

```
Enter choice: 6
Total Inventory Value: 444
```

Change Product :-

```
Enter choice: 7
Enter ID: 102
Enter new name: Apple
Enter new price: 101
Product details updated!
```

Exit :-

```
Enter choice: 0
Exiting...
```

Conclusion:-

The Inventory Management System using Hash Map successfully demonstrates the practical application of data structures in solving real-world problems. The project provides an efficient way to manage product records by allowing fast insertion, deletion, searching, and updating of inventory details. By using hashing, the system achieves better performance with reduced time complexity compared to traditional methods. It also includes useful features such as billing, low stock alert, and total inventory value calculation, making it more practical and reliable. This project not only improves understanding of hash tables and their implementation but also shows how data structures can be used to build scalable and efficient management systems.

Scope of Improvement:-

- ✓ A graphical user interface (GUI) can be added to make the system more user-friendly and interactive.
- ✓ Database integration such as MySQL can be used for permanent storage of product records instead of temporary memory storage.
- ✓ Additional features like user login, supplier management, and sales reports can be included for better functionality.
- ✓ The project can be converted into a web-based or mobile application for easier access and real-world business use.

Thank You